

Iftier Rahman

rahman.iftier@gmail.com | linkedin.com/in/iftier-rahman | +1 (416) 464 - 7184 | github.com/iftier23 | iftier-rahman.vercel.app

EDUCATION

University of Toronto

Toronto, ON

Bachelor of Applied Sciences in Computer Engineering, PEY Co-op

Anticipated Graduation Date: April 2027

Relevant Courses: Digital Systems, Operating Systems, Linear Algebra, Calculus, Software Design and Development, Object-Oriented Programming, Data Structures and Algorithms, Computer Organization, Applied Deep Learning

Awards: Recipient of Amazon Future Engineer Scholarship worth \$30,000

EXPERIENCE

Shopify

May 2025 – August 2025

Applied Machine Learning Software Engineer Intern

Toronto, ON

- Re-architected a multi-agent LLM assistant using **Python, Ruby, TypeScript** and **Langchain** into a tool-centric system with concurrent tool calls and agentic RAG, delivering **40%** latency improvement for **100k+** merchants.
- Built a **RAG-based caching layer** with **semantic retrieval** and **direct-match caching** using **Redis** and **Solr**; optimized with partitioning and localized indexes for global support, cutting time-to-first-token by **75%**.
- Resolved a data pipeline issue mislabeling **96%** of chatbot users by fixing **SQL** logic and **dbt** transformations in the **data warehouse**, orchestrated via an **Airflow DAG**, restoring accurate product attribution.
- Collaborated with cross-functional teams and engineers to iteratively ship improvements across production assistant infrastructure and architecture.

Amazon Web Services

May 2024 – August 2024

Software Development Engineering Intern

Vancouver, BC

- Architected and scaled a distributed workflow orchestration using **AWS Step Functions**, running up to **3,000 concurrent Lambdas** to backfill **350+ million schedules** in DynamoDB with new field across multiple regions
- Deployed and automated workflow orchestration infrastructure using **AWS CDK** and **CloudFormation** in **Typescript**, while leveraging **CloudWatch alarms and logs** to monitor updates, throttling, and failures
- Conducted **canary testing** in different environments using **TestNG** and **JUnit** and implemented robust security and permissions for internal **Java REST API**, ensuring secure, reliable performance across all environments.
- Authored a **comprehensive runbook** documenting the project, including troubleshooting steps, best practices, and monitoring strategies for **on-call engineers**

Amazon Web Services

May 2023 – August 2023

Software Development Engineering Intern

Vancouver, BC

- Developed internal **Java REST API** to list schedules by target for **AWS Eventbridge Scheduler**, utilizing a new target field for **customers** to efficiently retrieve schedules by target.
- Optimized **DynamoDB** data retrieval with a **Global Secondary Index (GSI)** based on the target field for targeted schedules, leading to a **30x** performance increase over filtered scan API calls
- Conducted **unit testing** using **JUnit** and **Mockito** for the internal **Java REST API**, achieving **98% code coverage** and ensuring validation of the API functionality
- Updated existing **REST APIs** to ensure created and updated schedules are populated with the target field in **DynamoDB**, enabling **100%** schedule retrieval of new schedules by the internal API

University of Toronto

January 2023 - April 2023

Team lead - APS112 Engineering Strategies and Practice 2

Toronto, ON

- Directed 6 member team** in optimizing high call volumes for Sunnybrook Academic Family Health Team
- Proposed MVP for email and online bookings, boosting patient processing by **70%** over current methods

TECHNICAL SKILLS

Languages: Java, Python, C, C++, JavaScript, Typescript, HTML, CSS, XML, Verilog, Assembly, Bash, Z shell

Tools: AWS, Git, FPGA, CI/CD, Agile, Firebase, APIs, NodeJS, OpenCV, Linux, Scripting, Windows PowerShell

Libraries/Frameworks: JUnit, Mockito, TestNG, NumPy, Matplotlib, Pandas, ReactJS, Pytorch, CUDA

PROJECTS

ReLive (Devpost) | *JavaScript, Firebase, Cohere API, OpenAI API, Google Cardboard*

- Built a VR photo album for UofTHacks 11 enabling 360° exploration of memories with Panolens.js and Google Cardboard, including API-driven song recommendations based on photo ambiance.

Diabetic Retinopathy Classification (GitHub) | *Python, PyTorch, OpenCV*

- Utilized **CUDA** on **NVIDIA L4** GPU to accelerate data-cleaning training, and validation on **40,000+** images
- Applied ResNet-152 via transfer learning with custom fully connected layers & dropouts, achieving **84% accuracy**

Pop-up Vaccine Clinic Locator (Link) | *HTML, CSS, Javascript, Figma*

- Over **500+** users, built using **HTML/CSS/Javascript**, designed UI using **Figma**
- Created based on personal experience with long-lines, short-notice of available clinics, & community consultations
- Users enter the first three digits of postal code and are presented with all eligible pop-up and hospital clinics

RookieXplore (GitHub) | *C++, OpenStreetMap API, GTK*

- Led a 3-member team to develop a map application using the **OpenStreetMap API**
- Optimized routing algorithms with **A***, **Dijkstra's**, and **Multi-Dijkstra**, achieving a **50% reduction in route calculation time** and **30% faster performance** through multithreading.
- Applied **Object-Oriented Programming** principles to structure the codebase for maintainability.

Fioscope (Link) | *Java, JavaScript, HTML, CSS, Android Studio*

- **Open-source** 3D-printed low-cost digital stethoscope based on the Glia project, reducing costs by **80%-90%** compared to traditional stethoscopes.
- Built a working prototype for **\$25** and developed an Android app in **Java**, enabling medical professionals to use Fioscope to listen, record, and save auscultations via a headphone jack.
- Developed a website providing instructions, source files hosted on GitHub, and resources for building Fioscope.

Super Mario Bros Parody (Link) | *C, FPGA, Nios 2 Processor*

- Developed a multi-level Super Mario Bros parody with custom characters on the **DE1-SoC FPGA board**
- Programmed audio and game mechanics using **C**, leveraging double buffering, audio FIFO, and PS/2 keyboard

Ping-Pong | *Verilog, FPGA*

- Developed the classic multi-player Pong game on the **DE1-SoC FPGA board** in **verilog**